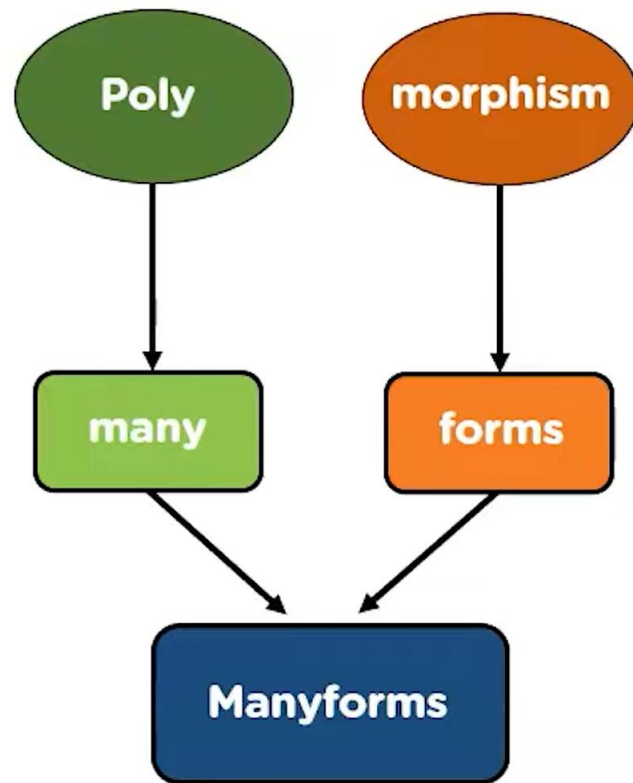


What's in it for you ?

- ▶ **What is Polymorphism in C++ ?**
- ▶ **Types of polymorphism**
- ▶ **Compile-time polymorphism**
- ▶ **Function overloading**
- ▶ **Operator overloading**
- ▶ **Runtime polymorphism**
- ▶ **Function overriding**
- ▶ **Virtual function**



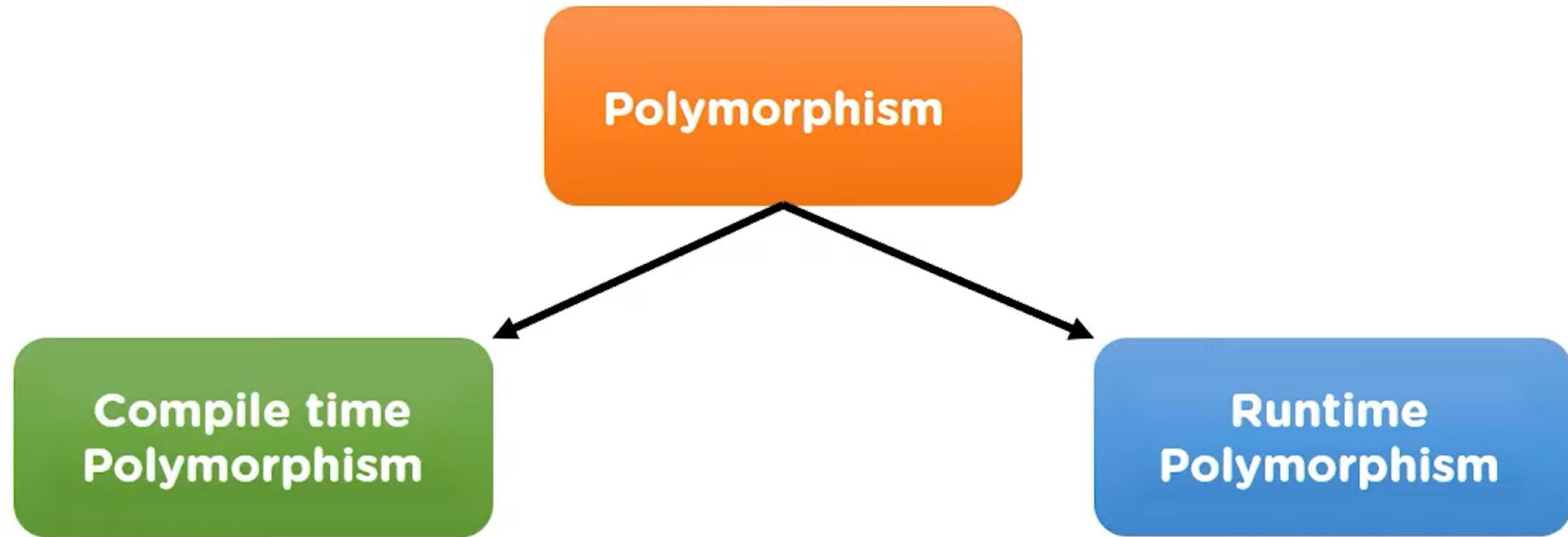
What is Polymorphism in C++?



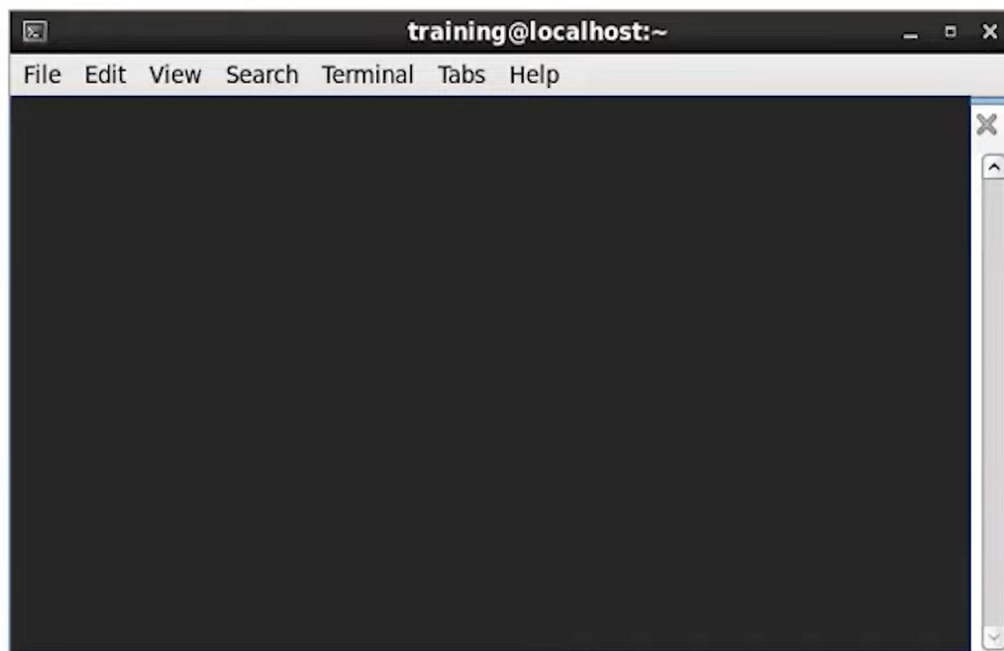
Polymorphism is derived from two words, i.e., **poly** and **morphism** where poly means many and morph means forms.

When a member function or an object behaves in different manner under different circumstances, then it can be defined as Polymorphism.

Types of polymorphism



Compile time polymorphism



As the name suggests, this type of polymorphism is achieved during the compile time.

This compile time polymorphism can be implemented using **function overloading** and **operator overloading**.

Function overloading in C++



Function overloading is a feature in C++ where functions having same name, but different types parameters can be overloaded.

Example:

```
Func ( int data1, int data2 );  
Func ( double data1, double data2 );  
  
Func ( int data1, int data2 );  
Func ( int data1, int data2, int data3 )
```

Function Overloading

When we have two functions with the same name but different parameters, different functions are called depending on the number and data types of parameters.

This is known as function overloading

Function Overloading can be achieved through the following two cases:

The names of the functions and return types are the same but differ in the type of arguments.

The name of the functions and return types are the same, but they differ in the number of arguments.

poly1.cpp •

▷ □ ...

C:\Users>harsh>Desktop>Work>poly1.cpp>main()

```
1  #include<iostream>
2  using namespace std;
3
4  class King
5  {
6      public:
7
8      void Pro(int a, int b)
9      {
10         cout<<a<<b<<endl;
11     }
12
13     void Pro(int a, int b, int c, int d, int e)
14     {
15         cout<<(a+b+c+d+e);
16     }
17 };
18
19 int main()
20 {
21     King k;
22     k.Pro(10, 20);
23     k.Pro(12, 23, 20, 62, 6);
24     return 0;
25 }
```

Operator overloading in C++



Just like functions, we can also overload operators in C++.

In operator overloading, we can redefine the functionality of the operator i.e., we can change how it works.

Example:

```
sum= s1.add(s2);
```

```
sum=s1+s2;
```

Operator Overloading

- A user-defined type must be present in at least one operand.
- ".", "::", typeid, size, ".*", and C++'s single ternary operator, "?:", are among the operators that cannot be overloaded.

These are some of the operators that can be overloaded in C++ :

Arithmetic operators : - , + , / , * , % and -= , += , /= , *= , %=

Boolean algebra : != , == , > , < , >= , <= , && , ||

Bit manipulation : & , | , ^ , << , >> and |= , &= , >>= , <<= , ^=

Memory management : new[] , new , delete[] , delete

```

#include <iostream>
using namespace std;

class Square_Box
{
private:
    float length;
    float width;
    float height;

public:
    Square_Box() {}
    Square_Box(float l, float w, float h)
    {
        length = l;
        width = w;
        height = h;
    }

    // Operator Overloading - "++" prefix operator
    void operator ++ ()
    {
        length++;
        width++;
        height++;
    }

    // Operator Overloading - "++" postfix operator
    void operator ++ (int)
    {
        length++;
        width++;
        height++;
    }

    void output()
    {
        cout << "\\tLength = " << length << endl;
        cout << "\\twidth = " << width << endl;
        cout << "\\tHeight = " << height << endl;
        cout << endl;
    }
};

```

```

int main()
{
    Square_Box mySquare_Box1(3.0, 5.0, 6.0);

    cout << "Dimensions of mySquare_Box1 = " << endl;
    mySquare_Box1.output();

    mySquare_Box1++;

    cout << "Dimensions of mySquare_Box1 = " << endl;
    mySquare_Box1.output();

    ++mySquare_Box1;

    cout << "Dimensions of mySquare_Box1 = " << endl;
    mySquare_Box1.output();

    return 0;
}

```

```
#include <iostream>
using namespace std;
//declaring the class that will overload the operator
class Count
{
    //declaring the variable required for the program
private:
    int i;

public:

    // Creating a Constructor to initialize count to 8
    Count() : i(8) {}

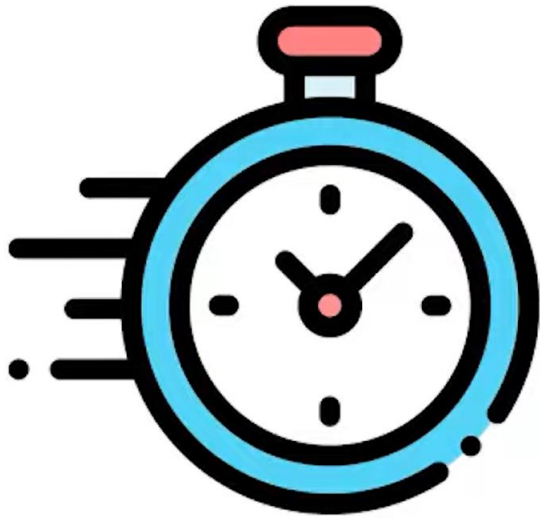
    // Overloading the ++ when used as prefix
    void operator ++ () {
        ++i;
    }
    //printing the data afteroverloading the ++ operator
    void display()
    {
        cout << "Count: " << i << endl;
    }
};

int main() {
    Count count1;

    // Call the "void operator ++ ()" function
    ++count1;

    count1.display();
    return 0;
}
```

Runtime Polymorphism



This type of polymorphism is achieved when the functions are called or invoked during the runtime.

It can be achieved through function overriding.

Runtime Polymorphism

- Polymorphism occurs when there is **a hierarchy of classes** and **they are related by inheritance**.
- C++ polymorphism means that a call to a member function will cause a **different function to be executed** depending on the type of **object that invokes the function**.

Runtime or Dynamic Polymorphism

- In runtime polymorphism, the compiler resolves the object at run time
- it decides which function call should be associated with that object.
- It is also known as dynamic or late binding polymorphism.
- This type of polymorphism is executed through **virtual functions and function overriding**.
- **All the methods** of runtime polymorphism get invoked during **the run time**.

Function Overriding

- Function overriding takes place when your derived class and base class both contain a function having the same name.
- Along with the same name, both the functions should have the same number of arguments as well as the same return type.
- The derived class inherits the member functions and data members from its base class.
- To override a certain functionality, you must perform function overriding.
- It is achieved during the run time.

Function overriding

```
Class parent
{
    void message()
    {
        cout<<"Hello there!";
    }
};

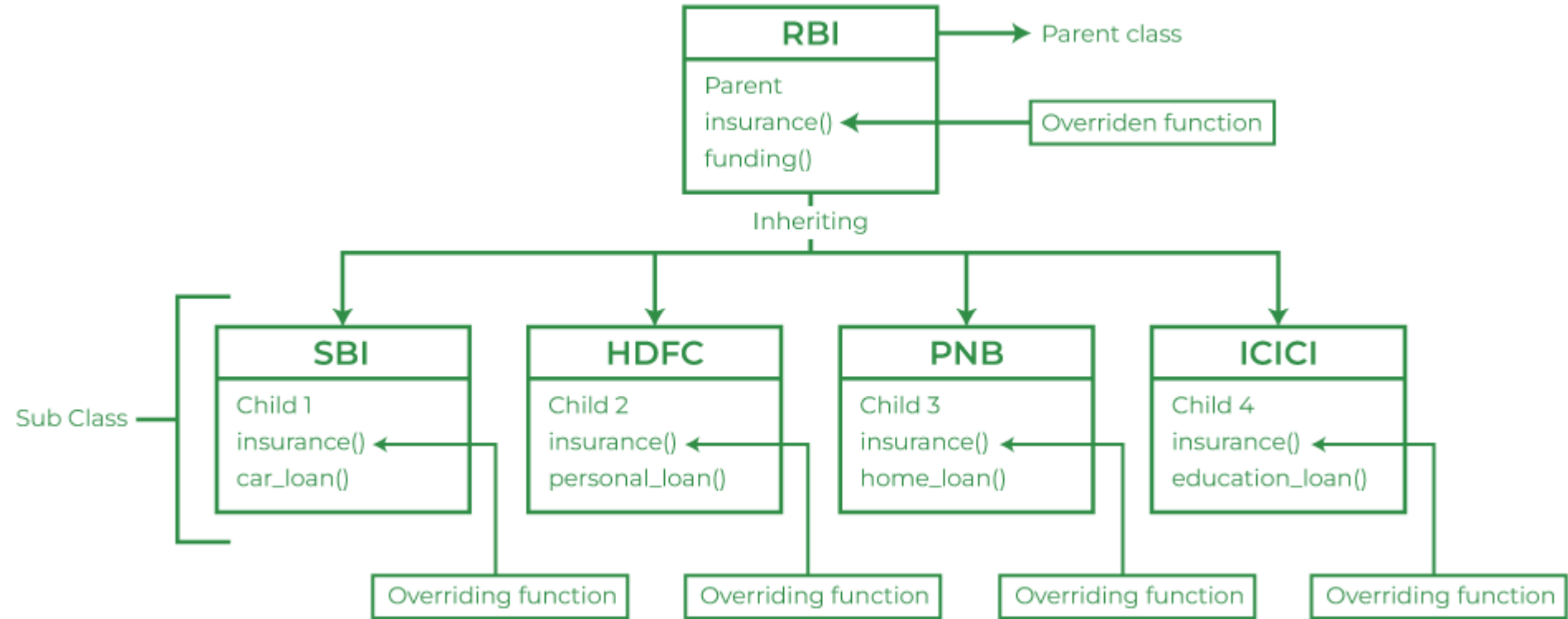
Class child : public parent
{
    void message()
    {
        cout<<"Nice to meet you!";
    }
};
```

In function overriding, the function of base class gets a new definition in the derived class.

The functions with same name have different definitions in the parent class and child class.

Function Overriding

- Functions that are overridden acquire different scopes.
- For function overriding, inheritance is a must. It can only happen in a derived class.
- If a class is not inherited from another class, can not achieve function overriding.



```
#include <iostream>

using namespace std;

class Account {
    public:
    float salary = 60000;
};

class Programmer: public Account {
    public:
    float bonus = 5000;
};

int main(void) {
    Programmer p1;
    cout<<"Salary: "<<p1.salary<<endl;
    cout<<"Bonus: "<<p1.bonus<<endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;
class bird
{
    public:

    void display()

    {
        cout << "I am the display function of the base class";
        cout << "\n\n";

    }

};
```

```
class parrot:public bird
{
    public:
    void display()
    {
        cout << "I am the display function of the derived class";
        cout << "\n\n";
    }

};

int main()
{
    bird b;
    parrot p;
    b.display();
    p.display();
}
```

I am the display function of the base class

I am the display function of the derived class

Virtual function

```
Class parent
{
    virtual void message()
    {
        cout<<"Hello!";
    }
};

Class child : public parent
{
    void message()
    {
        cout<<"Hi";
    }
};
```

Another way for implementing runtime polymorphism in C++ is **Virtual function**.

. It can be defined as the member function of base class and is redefined in the derived class.

It is declared with the keyword **virtual**.

```
#include<iostream>
using namespace std;
class B {
public:
    virtual void s() {
        cout<<" In Base \n";
    }
};
class D: public B {
public:
    void s() {
        cout<<"In Derived \n";
    }
};
int main(void) {
    D d; // An object of class D
    B *b= &d; // A pointer of type B* pointing to d
    b->s(); // prints "D::s() called"
    return 0;
}
```

/tmp/9lztmipgI5.o

In Derived

Compile Time Polymorphism	Run time Polymorphism
In Compile time Polymorphism, the call is resolved by the compiler.	In Run time Polymorphism, the call is not resolved by the compiler.
It is also known as Static binding, Early binding and overloading as well.	It is also known as Dynamic binding, Late binding and overriding as well.
Method overloading is the compile-time polymorphism where more than one methods share the same name with different parameters or signature and different return type.	Method overriding is the runtime polymorphism having the same method with same parameters or signature but associated with compared, different classes.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.
It provides fast execution because the method that needs to be executed is known early at the compile time.	It provides slow execution as compare to early binding because the method that needs to be executed is known at the runtime.
Compile time polymorphism is less flexible as all things execute at compile time.	Run time polymorphism is more flexible as all things execute at run time.
Inheritance is not involved.	Inheritance is involved.